

INTERSYSTEMS PT DEVELOPER COMMUNITY 2026 — PROGRAMMING CONTEST

IRIS Production Guardian

Copiloto de observabilidade, investigação de incidentes e consulta a documentação para administradores e desenvolvedores IRIS — construído inteiramente sobre InterSystems IRIS.

Documentação completa do projeto — versão 13/09/2026

Repositório: github.com/sergiofsq/iris-production-guardian

Autor/proprietário: Sergio Fernandes · Desenvolvimento assistido por IA (Claude Code)

Sumário

- 1 Sumário executivo
- 2 Visão geral e objetivos
- 3 Restrições tecnológicas
- 4 Arquitetura
- 5 Módulos principais
- 6 Funcionalidades bônus
- 7 Instalação e configuração
- 8 Testes e evidências
- 9 Segurança e auditoria de segredos
- 10 Metodologia de desenvolvimento com IA
- 11 Cronograma e status do projeto
- 12 Referências

1. Sumário executivo

IRIS Production Guardian é uma aplicação construída inteiramente sobre InterSystems IRIS (ObjectScript, interoperabilidade, Vector Search e Foreign Tables nativos — sem banco de dados externo) que observa uma Production de interoperabilidade em tempo real, investiga incidentes com evidência real coletada automaticamente e responde perguntas sobre a documentação do projeto com citação de fonte e abstenção calibrada quando não há informação suficiente.

O projeto nasceu como submissão para o Concurso de Programação da Comunidade de Desenvolvedores da InterSystems PT 2026, com prazo de submissão em 21/09/2026. Este documento consolida, em um único lugar, o que foi construído, como foi verificado e o que ainda falta — cada afirmação aqui corresponde a um teste real executado, não a uma suposição de como o sistema deveria se comportar.

Estado em 13/09/2026: os três módulos do MVP (Monitor, Investigator, RAG Assistant) e os quatro itens de bônus (Business Rules, busca híbrida, acesso a API pública, multimodelo) estão implementados e testados. Os quatro caminhos formais exigidos pela Definition of Done (percurso saudável, falha de destino, falta de dados, indisponibilidade de modelo) foram testados ao vivo. Uma instalação a partir de checkout limpo foi verificada — seis lacunas reais de documentação foram encontradas e corrigidas no processo. Faltam apenas conteúdo de entrega (vídeo de demonstração e artigo da comunidade), não engenharia.

2. Visão geral e objetivos

A aplicação resolve um problema real de operação: um administrador de uma Production IRIS precisa saber, a qualquer momento, (a) o que está saudável e o que não está, (b) por que um componente específico está com problema, com evidência e não com adivinhação, e (c) como resolver, com base na documentação real do projeto — sem depender de conhecimento tribal guardado na cabeça de uma pessoa só.

2.1 Production Monitor

Painel que mostra o estado real de cada componente (Service, Process, Operation) de uma Production: saudável, degradado, indisponível ou desconhecido, com contagem de fila, mensagens processadas e erros recentes, atualizado a cada 10 segundos. Persiste uma série temporal mínima e detecta hosts "travados" (fila positiva que não esvazia).

2.2 AI Incident Investigator

Dado um componente e uma janela de tempo, coleta evidência real (saúde atual, eventos com erro real, documentação relevante do RAG) e pede a um modelo de IA um resumo estruturado: observações, hipóteses separadas de fato observado, lacunas de evidência e próximos passos sugeridos. É somente leitura — nenhuma remediação é aplicada automaticamente.

2.3 RAG Assistant

Responde perguntas sobre o projeto usando um corpus de documentação indexado (busca híbrida vetorial + lexical), sempre citando a fonte, e recusando responder ("não sei") quando a similaridade com a base é baixa, em vez de inventar. Disponível com dois provedores de IA independentes para a etapa de geração (ver seção 6.4).

3. Restrições tecnológicas (inegociáveis)

- **InterSystems ObjectScript (COS) em primeiro lugar** para lógica de negócio, interoperabilidade e orquestração — nenhum framework externo de aplicação.
- **InterSystems IRIS é o único banco de dados.** Sem PostgreSQL, SQLite, MongoDB, Redis ou vector database externo — Vector Search e Foreign Tables nativos do IRIS cobrem essa necessidade.
- **Python só onde COS comprovadamente não resolve**, com a limitação documentada caso a caso. Na prática, nenhuma linha de Python foi necessária no projeto final — tudo roda em COS puro.
- **Interface web:** HTML/CSS simples e o mínimo de JavaScript de apresentação (spinners de carregamento, toggle de tema/idioma); toda regra de negócio permanece no backend COS.

4. Arquitetura

```
Web UI: Monitor | Investigator | RAG Assistant (Gemini) | RAG Assistant (Groq)
      |
Camada de aplicação: consultas, investigação, recuperação híbrida
      |
InterSystems IRIS: persistência e interoperabilidade
      |           |           |
Production COS   Eventos/métricas   Documentos/índices vetoriais
      |           |           |
      |           |           | (Vector Search + iFind nativos)
Service -> Process -> Operation
      |           |           |
      |           |           | (roteamento por Business Rule, severidade)
      |           |           | +-> destino padrão / destino prioritário
      |           |           |
Segunda Production independente: PublicHealth (bônus, API pública real)
```

Duas Productions independentes rodam no mesmo namespace (`GUARDIAN`): a principal (incidentes sintéticos, Service→Process→Operation) e uma segunda dedicada ao bônus de API pública, que faz polling real de um serviço externo sem chave de autenticação.

5. Módulos principais — detalhe técnico

5.1 Production COS

Classe	Papel
<code>Guardian.Messages.IncidentEvent</code>	Mensagem (<code>Ens.Request</code>) representando um incidente sintético
<code>Guardian.Service.FileIncidentService</code>	Business Service, <code>EnsLib.File.InboundAdapter</code> , lê incidentes de arquivos <code>.txt</code>
<code>Guardian.Process.IncidentRouterProcess</code>	Business Process em código COS (não BPL)
<code>Guardian.Operation.FileOutputOperation</code>	Business Operation, destino padrão (severidade LOW)
<code>Guardian.Operation.PriorityOutputOperation</code>	Business Operation, destino prioritário (HIGH/CRITICAL/MEDIUM)
<code>Guardian.Production.GuardianProduction</code>	Conecta os hosts acima

Cenário de falha/recuperação real e reproduzível: permissão de escrita do diretório de destino removida via `chmod` (falha real de sistema operacional, não simulada), erro `#5005` capturado, `FailureTimeout` do framework observado expirando, recuperação automática confirmada para tráfego novo, reenvio manual via `Ens.MessageHeader.ResendMessage` para a mensagem que falhou durante a janela de indisponibilidade.

5.2 Production Monitor

`Guardian.Monitor.StatusCollector` consulta classes `Ens.*` reais (nunca valor fixo) para saúde/fila/mensagens/erros de cada host; `Guardian.API.MonitorAPI` expõe isso como JSON; `Guardian.UI.MonitorPage` renderiza os cartões e persiste amostras em `Guardian_Monitor.HealthSample` a cada carregamento.

5.3 AI Incident Investigator

`Guardian.Investigator.EvidenceCollector` junta saúde atual, eventos reais da janela (com `$System.Status.GetErrorText()`, não o `%Status` bruto) e documentação relevante recuperada do RAG. `Guardian.Investigator.Analyzer` monta o prompt e chama o modelo de IA, separando explicitamente observação de hipótese. Cada investigação é persistida em `Guardian_Investigator.Investigation`.

5.4 RAG Assistant

Esquema (`Guardian.RAG.Schema`): tabelas `Guardian_RAG.Document` e `Guardian_RAG.Chunk`, com coluna `VECTOR(DOUBLE, 768)` nativa do IRIS. Ingestão (`Guardian.RAG.Ingestion`) faz chunking (800 caracteres, sobreposição de 150) e gera embeddings reais via API do provedor. Recuperação híbrida (`Guardian.RAG.Query.Retrieve`): busca vetorial (similaridade de cosseno, `VECTOR_COSINE`) fundida por Reciprocal Rank Fusion com busca lexical (índice `%iFind.Index.Basic` nativo). Geração (`Guardian.RAG.Query.Ask`) exige citação de fonte no prompt e abstém quando a melhor similaridade fica abaixo de um limiar calibrado (0,58) — a etapa de geração aceita qualquer client que implemente `Generate(prompt)`, permitindo trocar de provedor sem duplicar a lógica de recuperação.

6. Funcionalidades bônus

6.1 Business Rules **BÔNUS +2**

`Guardian.Rule.IncidentRoutingRule` (`Ens.Rule.Definition`) decide o destino do incidente pela severidade, editável visualmente no Rule Editor do Management Portal, sem recompilar o Process. Severidades HIGH/CRITICAL/MEDIUM vão para `PriorityOutputOperation` (diretório físico separado, `/durable/guardian/out_priority`); as demais para `FileOutputOperation` .

6.2 Busca híbrida **BÔNUS +3**

Recuperação do RAG combina similaridade vetorial pura com um índice de texto completo nativo do IRIS (iFind), fundidos por Reciprocal Rank Fusion (k=60). O limiar de abstenção continua baseado exclusivamente na similaridade vetorial pura, para não invalidar a calibração já testada.

6.3 Acesso a API pública real — PublicHealth **BÔNUS +2**

Segunda Production independente (`Guardian.Production.PublicHealthProduction`) fazendo polling real de `disease.sh/v3/covid-19/all` (API pública sem chave), via um `Ens.InboundAdapter` customizado. Padrão honesto de resiliência: sucesso grava linha "ao vivo"; falha cai para o último valor real conhecido, rotulado como cache com idade calculada e o status HTTP que causou a queda; sem cache disponível, marca indisponibilidade — nunca inventa número. Testado com falha forçada real (endpoint trocado para caminho inválido, HTTP 404 real).

6.4 Multimodelo — segundo provedor de IA real **BÔNUS**

Segundo provedor de geração wireado de verdade: **Groq** (`Guardian.RAG.GroqClient` , modelo `openai/gpt-oss-120b` , API compatível com o formato Chat Completions da OpenAI), free tier sem cartão de crédito. A recuperação/embeddings continuam no provedor principal (Gemini) — o Groq não oferece endpoint de embeddings no free tier, confirmado antes de decidir a arquitetura, não presumido. Testado de ponta a ponta pela página real: investigação → pergunta sugerida → resposta real, citada, gerada pelo Groq.

7. Instalação e configuração

Passos completos e verificados em `README.md` do repositório (seção "Running it locally"). Resumo:

1. Subir IRIS Community Edition em Docker (imagem `intersystems/iris-community:latest-cd` — não usar a variante `-arm64`, licença Community expirada nessa imagem).
2. Criar namespace `GUARDIAN` /database `GUARDIANDB` com interoperabilidade habilitada, e um usuário administrativo.
3. Compilar as classes de `src/Guardian/` e criar as tabelas SQL chamando as 4 classes `*.Schema.Setup()` (RAG, Monitor, Investigator, PublicHealth) — não acontece automaticamente ao compilar.
4. Criar os diretórios de runtime dos adaptadores de arquivo e iniciar a Production.
5. Servir os assets estáticos (logo, CSS, spinner) copiando-os para o diretório físico da aplicação CSP.
6. Configurar SSL de saída e as credenciais de IA (Gemini e/ou Groq) em `Ens.Config.Credentials` — nunca em código-fonte.
7. Ingerir o corpus de documentação do RAG.

Uma instalação a partir de checkout limpo foi verificada ao vivo em 13/09/2026, num container e volume Docker descartáveis, sem nenhum atalho do ambiente já configurado. Seis lacunas reais de documentação foram encontradas e corrigidas nesse processo — ver seção 8.4.

8. Testes e evidências

A Definition of Done do projeto exige evidência real, testada, para cada afirmação — não planos, stubs ou mocks. Os quatro caminhos formais abaixo foram testados ao vivo em 13/09/2026.

8.1 Percurso completo

Mensagem de severidade LOW enviada via arquivo, fluiu Service → Process → Operation, arquivo de saída gerado em ~10 segundos. Monitor confirmado via requisição HTTP real (login com usuário real): estado `Running`, os 4 hosts `healthy`.

8.2 Falha de destino

Permissão de escrita do diretório de destino removida via `chmod` real. Erro real capturado: `ERROR #5005: Cannot open file '/durable/guardian/out_priority/incident_INC-FASE5-004.txt'`, embrulhado em `<Ens>ErrFailureTimeout`. Estado do componente mudou para `degraded` no Investigator. Corrigida a permissão: tráfego novo entregue automaticamente; a mensagem que falhou não é reentregue sozinha (comportamento do framework) — reenviada manualmente via `Ens.MessageHeader.ResendMessage`, confirmada no destino.

Achado real durante este teste: o roteiro documentado usava severidade CRITICAL apontando para o diretório de destino antigo — desatualizado pelo bônus de Business Rules (seção 6.1), que passou a rotear essa severidade para um destino diferente. Quebrar o diretório antigo não causava falha nenhuma. Corrigido o roteiro para o diretório correto.

8.3 Falta de dados

RAG Assistant com pergunta irrelevante ("qual a receita de bolo de chocolate?"): respondeu "não sei" em vez de inventar, mesmo com uma investigação recente em contexto. AI Incident Investigator, componente com 65 segundos sem nenhum tráfego, janela de 1 minuto: relatou corretamente "0 eventos registrados", sem inventar atividade.

8.4 Indisponibilidade de modelo de IA

Formalizado como caso determinístico (não dependente de esperar um erro de limite de uso acontecer por acaso): credencial real do Gemini trocada temporariamente por um valor inválido numa única sessão, erro real capturado (`HTTP 400: API key not valid`), credencial original restaurada antes de encerrar a sessão, confirmada funcionando novamente com uma chamada real depois.

8.5 Instalação a partir de checkout limpo

Container e volume Docker novos, seguindo só o README, sem nenhum atalho de configuração manual prévia. Seis lacunas reais encontradas e corrigidas: (1) tabela de status do README desatualizada; (2) "interoperabilidade habilitada" sem forma scriptável documentada; (3) diretórios de runtime nunca criados automaticamente; (4) passo de assets estáticos incompleto — um arquivo

(spinner de carregamento) nunca tinha sido versionado no repositório; (5) nenhuma menção a SSL/credenciais de IA; (6) as 4 chamadas de setup de esquema SQL não documentadas. Aplicação completa (Monitor, Investigator, RAG com ingestão e citação reais) confirmada funcionando de ponta a ponta no ambiente novo antes de destruí-lo.

9. Segurança e auditoria de segredos

Auditoria completa realizada em 13/09/2026, cobrindo não só o estado atual do repositório mas **todo o histórico do git** — relevante porque o repositório é público, e um segredo já removido continuaria exposto em commits antigos.

- Nenhuma chave de API ou senha real encontrada em nenhum commit já feito (busca por padrão em todo o histórico, não só no HEAD atual).
- Credenciais (Gemini, Groq) sempre lidas em tempo de execução de `Ens.Config.Credentials`, nunca em código-fonte — placeholders explícitos (`<CHAVE_REAL_AQUI>`) em toda a documentação de setup.
- Pasta pessoal com uma senha em texto puro confirmada como **nunca rastreada** pelo git desde o primeiro commit.
- Achado de privacidade (não segurança): screenshots usados no manual do usuário expunham a barra de favoritos do navegador do proprietário. Corrigido recortando as imagens-fonte e as imagens já embutidas no `.docx`, sem alterar nada do que já era exibido ao ler o documento normalmente.

10. Metodologia de desenvolvimento com IA

O projeto foi construído com assistência de IA (Claude Code), seguindo um conjunto de regras estritas contra alucinação: nunca declarar algo testado sem execução real observada; nunca inventar classes, métodos ou resultados; registrar toda pendência técnica como `VERIFY_REQUIRED` até ter evidência; distinguir sempre planejado, implementado, testado e demonstrado.

A sequência real de prompts que reproduz esta aplicação — reconstruída por engenharia reversa a partir do código, dos commits e dos documentos de planejamento onde o prompt literal original não foi preservado — está publicada em `entregaveis/Prompts/` no repositório, um arquivo por fase/bônus, cada um pareando o prompt com os erros reais encontrados e corrigidos naquela etapa (não uma versão idealizada sem o atrito que de fato aconteceu). Exemplos reais de correção de alucinação/erro de IA documentados ao longo do projeto:

- Nome de modelo de IA desatualizado no conhecimento do agente, corrigido pela própria resposta de erro da API (aconteceu duas vezes, com dois provedores diferentes).
- Uso de API do IRIS que não existe (`%GetLastIdentity()`), corrigido usando `%ROWID` do resultset real.
- Falso alarme de encoding UTF-8 investigado e descartado com evidência de bytes reais (hexdump), em vez de "corrigir" um código que já estava certo.
- Suposição de forma de habilitar interoperabilidade via GUI sem equivalente scriptável — resolvida por tentativa e erro real contra o dicionário de classes do IRIS, não por documentação externa.

11. Cronograma e status do projeto

Período	Trabalho	Status em 13/09/2026
08–10/09	Fase 0 — ambiente, arquitetura	CONCLUÍDO
11–14/09	Fase 1 + Fase 2 — Production COS, Monitor	CONCLUÍDO, ADIANTADO
15–17/09	Fase 3 + Fase 4 — RAG, Investigator	CONCLUÍDO, ADIANTADO
18–19/09	Fase 5 — hardening, testes, instalação limpa	CONCLUÍDO, UMA SEMANA ADIANTADO
20/09	Vídeo, artigo, auditoria final	PENDENTE
21/09	Submissão	PENDENTE

Pendências abertas

- Vídeo de demonstração — não iniciado.
- Artigo da comunidade (obrigatório para submissão, em português) — não iniciado.
- Reingestão reproduzível de 5 dos 12 documentos do corpus RAG (páginas externas) — sem script registrado ainda.
- Dois itens `VERIFY_REQUIRED` dependentes da organização do concurso: horário/fuso oficial de corte da submissão; interpretação do teto de bônus.

Avaliação honesta: tecnicamente, todos os itens de engenharia da Definition of Done interna estão fechados com evidência real. O que resta é conteúdo de entrega (vídeo e artigo), não desenvolvimento — e o projeto está adiantado em relação ao cronograma original.

12. Referências

- Repositório: github.com/sergiofsq/iris-production-guardian
- [docs/planejamento/00_MASTER_PLAN.md](#) — plano mestre e registro cronológico de pendências
- [docs/planejamento/01-06_*.md](#) — documentos de cada fase, com evidência datada
- [docs/planejamento/07_SCORECARD_EVIDENCIAS.md](#) — scorecard de evidências por critério do concurso
- [docs/experiments/01_falha_recuperacao_producao.md](#) — roteiro reproduzível do cenário de falha/recuperação
- [entregaveis/Prompts/](#) — sequência de prompts de desenvolvimento com IA
- [entregaveis/Manual_IRIS_Production_Guardian.pdf](#) — manual de uso da aplicação